BİL377 - HAFTA 04

# Özyineleme ve Rekürsiyonlar

Çağrılar, dönüş değerleri ve adım adım maliyet analizi

## Alt klasörlerdeki dosyaları saymak

# Bir klasörde kaç dosya var?



```
Ders/
├── plan.pdf
├── Notlar/
│   ├── konu.pdf
│   └── Ornekler/
│       └── kod.py
└── Odevler/
    ├── soru.pdf
    └── cozum.py
```

## Alt klasörler dahil

Ders klasörünün içindeki bütün dosyaları saymak istiyoruz.

Klasör adları sayılmaz.

Beklenen sonuç: 5 dosya

**Her alt klasörde aynı soru var: İçinde, alt klasörleriyle birlikte kaç dosya bulunuyor?**

# Alt klasörden gelen sayıyı toplamak



Bir klasörün sonucu = doğrudan içindeki dosyalar + alt klasörlerin sonuçları.

| Klasör | Kendi dosyaları | Alt klasörlerden gelen | Sonuç |
|---|---|---|---|
| Ornekler | 1 (kod.py) | 0 | 1 |
| Notlar | 1 (konu.pdf) | Ornekler: 1 | 2 |
| Odevler | 2 (soru.pdf, cozum.py) | 0 | 2 |
| Ders | 1 (plan.pdf) | Notlar: 2, Odevler: 2 | 5 |

**Ders için 1 + 2 + 2 = 5. Her dosya yalnızca bulunduğu klasörde sayılır.**

# Klasörde dosya sayan fonksiyon



```
def dosya_say(klasor):
    toplam = 0
    for oge in klasor.iterdir():
        if oge.is_symlink():
            continue
        if oge.is_file():
            toplam += 1
        elif oge.is_dir():
            toplam += dosya_say(oge)
    return toplam
```

**Python: pathlib.Path**

iterdir(): içindekileri gez

is_file(): dosya mı?
is_dir(): klasör mü?

Dosya için 1 ekle.
Alt klasör için
dönen sayıyı ekle.

**dosya_say(oge), aynı işlemi o alt klasör için başlatır.**

# Çağrıların açılması ve sonuçların dönüşü



Bu izde Notlar, ardından Odevler geziliyor. İlk çağrının derinliği 0.

| Adım | Derinlik | Çağrı olayı | Açıklama |
|---|---|---|---|
| 1 | 0 | Ders açılır | Alt klasör sonuçlarını bekler. |
| 2 | 1 | Notlar açılır | Ornekler'in sonucunu bekler. |
| 3 | 2 | Ornekler açılır | kod.py sayılır. |
| 4 | 2 | Ornekler 1 döner | Notlar hesabı devam eder. |
| 5 | 1 | Notlar 2 döner | Ders hesabı devam eder. |
| 6 | 1 | Odevler açılır | İki dosya sayılır. |
| 7 | 1 | Odevler 2 döner | Ders hesabı devam eder. |
| 8 | 0 | Ders 5 döner | Sayım tamamlanır. |

**Sonuç 5 dosya. Toplam 4 çağrı. En büyük derinlik 2, aynı anda en çok 3 açık çağrı.**

# Bu örnekte özyineleme neden durur?



Her çağrı bir alt klasöre iner. Alt klasör kalmayınca yeni çağrı yapılmaz.

| Kavram | Klasör örneğinde | Sonuca etkisi |
|---|---|---|
| Alt problem | Bir alt klasördeki dosyaları saymak | Aynı iş daha küçük bir alt ağaçta. |
| Taban durum | Alt klasör yok | Dosyaları say ve doğrudan dön. |
| Birleştirme | Kendi dosyaları + alt sonuçlar | Her dosya toplamda bir kez yer alır. |
| Boş klasör | Dosya ve alt klasör yok | 0 döner. |

**Özyinelemeli çözüm: küçük problemin cevabını al, bulunduğun problemin hesabında kullan.**

# Faktöriyel: n! hesabı



```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))  # 24
```

## Tanım

0! = 1

n! = n × (n – 1)!

4! = 4 × 3 × 2 × 1

Girdi: n ≥ 0, tam sayı

**Taban değer 1’dir: çarpmada sonucu değiştirmeyen eleman.**

# Faktöriyel çağrıları açılırken



factorial(4) için her çağrı kendi n değerini ve bekleyen çarpmasını saklar.

| Derinlik | Çağrı | Bu çağrıda bekleyen işlem |
|---|---|---|
| 0 | factorial(4) | 4 × factorial(3) |
| 1 | factorial(3) | 3 × factorial(2) |
| 2 | factorial(2) | 2 × factorial(1) |
| 3 | factorial(1) | 1 × factorial(0) |
| 4 | factorial(0) | Bekleme yok: 1 döndür |

**Tabana ulaşıldığında factorial fonksiyonuna ait 5 açık çağrı vardır.**

# Faktöriyel sonuçları dönerken



Dönüş sırası, çağrıların açılma sırasının tersidir.

| Dönüş sırası | Tamamlanan çağrı | Yapılan hesap | Dönen değer |
|---|---|---|---|
| 1 | factorial(0) | Taban | 1 |
| 2 | factorial(1) | 1 × 1 | 1 |
| 3 | factorial(2) | 2 × 1 | 2 |
| 4 | factorial(3) | 3 × 2 | 6 |
| 5 | factorial(4) | 4 × 6 | 24 |

**return, hesaplanan değeri bir üst çağrıya taşır.**

# Sonuç, çağrı sayısı ve derinlik



factorial(n) için farklı soruların farklı cevapları vardır.

| n | Hesaplanan n! | Çağrı sayısı | Çarpma sayısı | En büyük derinlik |
|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 2 | 1 | 1 |
| 4 | 24 | 5 | 4 | 4 |
| 5 | 120 | 6 | 5 | 5 |

**factorial(4) = 24; bu, 24 çağrı yapıldığı anlamına gelmez.**

# Koddan çağrı bağıntısını çıkarma



C(n): factorial(n) çalışırken yapılan toplam factorial çağrısı.

| Kodda olan | Saydığımız çağrı | Bağıntıdaki karşılığı |
|---|---|---|
| n == 0 | Yalnızca bu çağrı | C(0) = 1 |
| n > 0 | İçinde olduğumuz çağrı | 1 |
| factorial(n – 1) | Alt çağrı ve onun devamı | C(n – 1) |
| Hepsi birlikte | Şimdiki + alt çağrılar | C(n) = 1 + C(n – 1) |

**Buradaki +1, bir çarpmayı değil, içinde bulunduğumuz çağrıyı sayıyor.**

# C(n) = 1 + C(n – 1) çözümü



Taban: C(0) = 1. Her açılımda bir çağrı daha sayıyoruz.

$$C(n) = 1 + C(n - 1)$$

$$C(n) = 2 + C(n - 2) = 3 + C(n - 3)$$

k açılım sonra: $C(n) = k + C(n - k)$

**$k = n \Rightarrow C(n) = n + C(0) = n + 1$**

**Kontrol: n = 4 için C(4) = 5; çağrı tablosuyla aynı.**

# Sabit yerel iş: T(n) = T(n – 1) + 2



Açıklama modeli: taban 1 birim, her pozitif çağrının kendi işi 2 birim.

$$T(0) = 1; \quad T(n) = T(n - 1) + 2$$

$$T(4) = T(3) + 2 = T(2) + 4$$

$$T(4) = T(1) + 6 = T(0) + 8 = 9$$

$$\mathbf{T(n) = T(n - k) + 2k \Rightarrow T(n) = 2n + 1}$$

**2 ve 1 bu modelin kabulüdür; Python süresi veya tüm satırların kesin sayımı değildir.**

# Doğrusal maliyette O, Ω ve Θ



$T(n) = 2n + 1$ için $n \geq 1$ alalım.

Alt sınır: $2n \leq 2n + 1$

Üst sınır: $2n + 1 \leq 3n$ (çünkü $1 \leq n$)

Birlikte: $2n \leq T(n) \leq 3n$

**$T(n) \in \Omega(n)$, $T(n) \in O(n)$, $T(n) \in \Theta(n)$**

**O: üst sınır. Ω (omega): alt sınır. Θ (teta): aynı büyüme için iki sınır birlikte.**

# Her çağrıda n iş yapılırsa



Bu kez alt problem yine n – 1; fakat çağrının kendi işi sabit değil.

$$W(0) = 0; \quad W(n) = W(n - 1) + n$$

$$W(4) = 4 + 3 + 2 + 1 + W(0) = 10$$

$$W(n) = n + (n - 1) + \ldots + 1 = n(n + 1)/2$$

$$\mathbf{n^2/2 \leq W(n) \leq n^2 \quad (n \geq 1) \Rightarrow W(n) \in \Theta(n^2)}$$

**Aynı n – 1 küçülmesi, farklı yerel iş nedeniyle farklı toplam maliyet verebilir.**

# Döngülü özyineleme: işlem sayımı



```
def is_sayisi(n):
    if n == 0:
        return 0
    sayac = 0
    for _ in range(n):
        sayac += 1
    return sayac + is_sayisi(n - 1)
```

**n = 4 için**

Çağrı sayısı kaç?

Döngü gövdesi toplam kaç kez çalışır?

Dönen değer nedir?

Zamanın sıkı sınırı?

**Ölçütleri ayrı sayın: fonksiyon çağrısı ve döngü gövdesi aynı şey değildir.**

# İkili arama: dizi ve indisler



Artan sıralı dizi: [2, 5, 8, 11, 14, 18, 21]. Aranan değer x = 14.

| İndis | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| Değer | 2 | 5 | 8 | 11 | 14 | 18 | 21 |
| Aralık | lo = 0 | | | mid = 3 | | | hi = 6 |

**Sözleşme: bulunursa bir indis, bulunamazsa –1 döndür. İki sınır da aralığa dahil.**

# Özyinelemeli ikili arama: tam kod



```
def binary_search(a, x, lo, hi):
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if a[mid] == x:
        return mid
    if x < a[mid]:
        return binary_search(
            a, x, lo, mid - 1)
    return binary_search(
        a, x, mid + 1, hi)
```

## İlk çağrı

binary_search(
a, 14, 0, len(a)-1)

Boş aralık: lo > hi

Sol: hi = mid – 1
Sağ: lo = mid + 1

**Bir çağrı ya sonuç döndürür ya da yalnızca bir alt aralıkta devam eder.**

# 14 aranırken gerçekleşen adımlar



a = [2, 5, 8, 11, 14, 18, 21], x = 14.

| Derinlik | [lo, hi] | Boyut | mid / a[mid] | Karar |
|---|---|---|---|---|
| 0 | [0, 6] | 7 | 3 / 11 | 14 > 11 → sağ |
| 1 | [4, 6] | 3 | 5 / 18 | 14 < 18 → sol |
| 2 | [4, 4] | 1 | 4 / 14 | Eşit → 4 döndür |

**Dönen indis: 4. Çağrı sayısı: 3. En büyük derinlik: 2.**

# 13 aranırken bulunamama durumu



Aynı dizide 13 yok. Boş aralığa ulaşana kadar devam ederiz.

| Derinlik | [lo, hi] | Boyut | Orta değer | Karar |
|---|---|---|---|---|
| 0 | [0, 6] | 7 | 11 | 13 > 11 → sağ |
| 1 | [4, 6] | 3 | 18 | 13 < 18 → sol |
| 2 | [4, 4] | 1 | 14 | 13 < 14 → sol |
| 3 | [4, 3] | 0 | Yok | lo > hi → −1 |

**4 çağrı vardır; son çağrı eleman karşılaştırmaz, boş aralığı kontrol eder.**

# İkili arama neden doğru ve neden durur?



Her çağrıda hedefin bulunabileceği aralığı korur, aralığı kesin olarak küçültürüz.

| Durum | Elenen bölüm | Yeni aralık |
|---|---|---|
| x = a[mid] | Eleme gerekmez | mid döner. |
| x < a[mid] | mid ve sağındaki değerler | [lo, mid – 1] |
| x > a[mid] | mid ve solundaki değerler | [mid + 1, hi] |
| lo > hi | Aday kalmamıştır | –1 döner. |

**Aralıkta kalan eleman sayısı azalır. Sıralılık, atılan bölümde hedef olmadığını garantiler.**

# İkili aramanın çağrı bağıntısı



C(n): n elemanlı aramada en kötü durumdaki çağrı sayısı; boş çağrı dahil.

$$C(0) = 1$$

$$C(n) = 1 + C(\lfloor n/2 \rfloor) \qquad (n \geq 1)$$

$$C(7) = 1 + C(3) = 2 + C(1)$$

$$\mathbf{C(7) = 3 + C(0) = 4}$$

**Tek alt çağrı olduğu için C(n/2) var; 2C(n/2) yok.**

# Yarılama ve logaritma



15 elemanlık en uzun çağrı zinciri: 15 → 7 → 3 → 1 → 0.

| Küçülme sayısı k | Kalan boyut | $2^k$ | Bu noktaya kadar çağrı |
|---|---|---|---|
| 0 | 15 | 1 | 1 |
| 1 | 7 | 2 | 2 |
| 2 | 3 | 4 | 3 |
| 3 | 1 | 8 | 4 |
| 4 | 0 | 16 | 5 |

**$2^4 = 16 > 15$: boş aralığa 4 küçülmede ulaşılır, ilk çağrıyla toplam 5 çağrı olur.**

# İkili aramanın genel çözümü



Başlangıç $n \geq 1$. $k$ küçülmeden sonra kalan boyut $\lfloor n/2^k \rfloor$ olur.

Tabana ulaşma: $\lfloor n/2^k \rfloor = 0 \Leftrightarrow 2^k > n$

En küçük tam sayı $k = \lfloor \log_2 n \rfloor + 1$

$C(n) = k + 1 = \lfloor \log_2 n \rfloor + 2$

**$C(n) \in \Theta(\log n)$ ($n \geq 2$ için büyüme)**

**Örnek: $n = 7 \Rightarrow 2 + 2 = 4$ çağrı; $n = 15 \Rightarrow 3 + 2 = 5$ çağrı.**

# İkili aramada zaman ve yardımcı bellek



Sıralı dizi hazırdır. Arama indislerle yapılır; alt diziler kopyalanmaz.

| Durum / uygulama | Zaman | Yardımcı bellek |
|---|---|---|
| Hedef ilk ortada | Θ(1) | Θ(1) |
| Özyinelemeli, en kötü | Θ(log n) | Θ(log n) |
| Döngülü, en kötü | Θ(log n) | Θ(1) |

**Bellek hesabında asıl girdi dizisini değil, aramanın ek olarak kullandığı alanı sayıyoruz.**

# Aynı aramanın döngülü sürümü



```
def binary_search_iter(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if x < a[mid]:
            hi = mid - 1
        else:
            lo = mid + 1
    return -1
```

## Aynı kararlar

14 için indis 4
13 için –1

Yeni çağrı açılmaz.
Sınırlar güncellenir.

Yardımcı alan: Θ(1)

**Özyinelemeli ve döngülü sürümler aynı aralıkları inceleyebilir.**

# Alt diziyi kopyalamak maliyeti değiştirir



Python listesinde a[:mid] yeni bir liste oluşturur; yalnızca sınır değiştirmek değildir.

| Uygulama | Çağrının kendi işi | En kötü zaman |
|---|---|---|
| Aynı liste + lo, hi | Sabit sayıda işlem | Θ(log n) |
| Liste dilimleme | Θ(n) referans kopyalama | Θ(n) |
| Kopyaların toplamı | n/2 + n/4 + … < n | Doğrusal büyüme |

**Yarılama tek başına logaritmik zaman garantilemez; her seviyedeki iş de sayılır.**

# Hızlı üs alma: üssü yarıya indirmek



$x^n$ için n negatif olmayan tam sayı olsun.

$x^0 = 1$

n = 2k (çift) ise: $x^n = (x^k)^2$

n = 2k + 1 (tek) ise: $x^n = x \times (x^k)^2$

**$2^{13} = 2 \times (2^6)^2 = 2 \times 64 \times 64 = 8192$**

**Alt sonucu bir kez hesapla, sakla ve çarpımda iki kez kullan.**

# Hızlı üs alma: çalışan kod



```
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    if n % 2 == 0:
        return half * half
    return x * half * half

print(power(2, 13))  # 8192
```

## Bir alt çağrı

n // 2: tam sayı bölme

n % 2: ikiye bölümden kalan

half: alt çağrıdan gelen sonuç

**half * half, fonksiyonu yeniden çağırmaz; eldeki iki değeri çarpar.**

# power(2, 13): çağrı ve dönüş değerleri



Çağrılar: 13 → 6 → 3 → 1 → 0. Sonuçlar alttan yukarı hesaplanır.

| n | half değeri | Dönüşte yapılan hesap | Dönen değer |
|---|---|---|---|
| 0 | Yok | Taban | 1 |
| 1 | 1 | 2 × 1 × 1 | 2 |
| 3 | 2 | 2 × 2 × 2 | 8 |
| 6 | 8 | 8 × 8 | 64 |
| 13 | 64 | 2 × 64 × 64 | 8192 |

**5 çağrı, en büyük derinlik 4; bu kodda toplam 7 çarpma.**

# Hızlı üs almanın maliyeti



C(n): sıfır tabanı dahil çağrı sayısı. Her çağrıda en fazla 2 çarpma.

$$C(0) = 1; \quad C(n) = 1 + C(\lfloor n/2 \rfloor)$$

$$n \geq 1 \text{ için } C(n) = \lfloor \log_2 n \rfloor + 2$$

$$n = 13 \Rightarrow C(13) = 3 + 2 = 5$$

**Zaman: $\Theta(\log n)$ • Yardımcı yığın: $\Theta(\log n)$**

**Zaman ifadesi, çarpma dahil aritmetik işlemlerin sabit maliyetli sayıldığı modele aittir.**

# Aynı alt sonucu iki kez hesaplamak



## Bir kez hesapla

```
half = power(x, n // 2)
return half * half
```

1 alt çağrı: C(n/2) + 1

## İki kez yeniden hesapla

```
return (power(x, n // 2)
        * power(x, n // 2))
```

2 alt çağrı: 2C(n/2) + 1

Burada yalnızca çift üs dalı gösteriliyor; taban ve tek üs dalı ayrıca korunur.

# İki alt çağrılı bir maliyet modeli



Yeni model: n, 2’nin kuvveti; her çağrı 1 birim; taban n = 1.

$$T(1) = 1; \quad T(n) = 2T(n/2) + 1$$

$$T(2) = 2 \times 1 + 1 = 3$$

$$T(4) = 2 \times 3 + 1 = 7$$

$$\mathbf{T(8) = 2 \times 7 + 1 = 15}$$

**Derinlik yalnızca 3 olsa da toplam 15 çağrı vardır.**

# Dallanma: toplam çağrı ve açık çağrı



$T(n) = 2T(n/2) + 1$, $T(1) = 1$; $n = 8$.

| Seviye | Çağrı başına boyut | Çağrı adedi | Seviyenin toplam işi |
|---|---|---|---|
| 0 | 8 | 1 | 1 |
| 1 | 4 | 2 | 2 |
| 2 | 2 | 4 | 4 |
| 3 | 1 | 8 | 8 |

**Toplam iş: 1 + 2 + 4 + 8 = 15. Sıralı yürütmede en çok 4 çağrı açıktır.**

# İki sıralı parçayı birleştirmek



Sol: [2, 5, 8, 11] • Sağ: [1, 4, 7, 10]. Baştaki küçük değer sırayla alınır.

| Adım | Solun başı | Sağın başı | Çıktıya eklenen |
|---|---|---|---|
| 1 | 2 | 1 | 1 |
| 2 | 2 | 4 | 2 |
| 3 | 5 | 4 | 4 |
| 4 | 5 | 7 | 5 |
| Devam | 8, 11 | 7, 10 | 7, 8, 10, 11 |

**Sonuç: [1, 2, 4, 5, 7, 8, 10, 11]. Toplam 8 eleman çıktıya aktarılır.**

# Birleştirmeli sıralamanın reküransı



n, 2’nin kuvveti olsun. İki yarı sıralanır, sonra n eleman birleştirilir.

| Aşama | Problem boyutu / iş | Maliyete katkı |
|---|---|---|
| Sol yarıyı sırala | n/2 eleman | T(n/2) |
| Sağ yarıyı sırala | n/2 eleman | T(n/2) |
| Birleştir | Toplam n eleman | Θ(n) |
| Toplam | İki alt problem + yerel iş | T(n) = 2T(n/2) + Θ(n) |

**Hesap modeli: T(1) = 1 ve T(n) = 2T(n/2) + n.**

# Rekürans ağacında seviye maliyetleri



$T(n) = 2T(n/2) + n$, $T(1) = 1$; $n = 8$.

| Seviye | Çağrı sayısı | Boyut | Çağrı başına iş | Seviye toplamı |
|---|---|---|---|---|
| 0 | 1 | 8 | 8 | 8 |
| 1 | 2 | 4 | 4 | 8 |
| 2 | 4 | 2 | 2 | 8 |
| 3 (taban) | 8 | 1 | 1 | 8 |

**Toplam: $8 + 8 + 8 + 8 = 32 = 8 \log_2 8 + 8$.**

# Aynı bağıntıyı açarak çözme



$T(1) = 1$, $T(n) = 2T(n/2) + n$; n, 2'nin kuvveti.

$$T(n) = 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n$$

$$T(n) = 8T(n/8) + 3n$$

$$k \text{ açılım: } T(n) = 2^k T(n/2^k) + kn$$

$$\mathbf{k = \log_2 n \Rightarrow T(n) = n + n \log_2 n \in \Theta(n \log n)}$$

**Ağaç hesabı ve cebir aynı sonucu verdi; n = 8 için 32.**

# Reküransları karşılaştırma



T(1) = 1; yarılama satırlarında n, 2'nin kuvveti. Modellerde yerel iş açıkça verilmiştir.

| Bağıntı | Alt çağrı | Yerel iş | Çözüm |
|---|---|---|---|
| $T(n) = T(n - 1) + 1$ | 1; bir küçülür | 1 | $n \Rightarrow \Theta(n)$ |
| $T(n) = T(n/2) + 1$ | 1; yarıya iner | 1 | $1 + \log_2 n \Rightarrow \Theta(\log n)$ |
| $T(n) = 2T(n/2) + 1$ | 2; yarıya iner | 1 | $2n - 1 \Rightarrow \Theta(n)$ |
| $T(n) = 2T(n/2) + n$ | 2; yarıya iner | n | $n + n \log_2 n \Rightarrow \Theta(n \log n)$ |

**Bağıntıyı belirleyen üç şey: alt çağrı sayısı, alt problem boyutu, çağrının kendi işi.**

# Hata incelemesi: neden çalışmıyor?



## A • Dönüş değeri kayboluyor

```
def f(n):
    if n == 0:
        return 1
    n * f(n - 1)
```

**f(1) ne döndürür?**
**f(2) ne olur?**

## B • Problem küçülmüyor

```
def g(n):
    if n == 0:
        return 0
    return 1 + g(n)
```

**g(1), tabana ulaşır mı?**
**Hangi satır değişmeli?**

**Bir fonksiyonun bitmesi, doğru değer döndürdüğünü tek başına göstermez.**

# Uygulama: bağıntı ve izleme



Her durumda neyi saydığınızı ve tabanı belirtin.

| Problem | İstenen | Sonuç kontrolü |
|---|---|---|
| factorial(5) | Çıktı / çağrı / derinlik | 120 / 6 / 5 |
| Aynı dizide 21'i ara | Aralıkları ve indisi bul | [0,6] → [4,6] → [6,6]; indis 6 |
| power(3, 5) | Çağrı zinciri ve sonuç | 5 → 2 → 1 → 0; sonuç 243 |
| T(1)=1, T(n)=2T(n/2)+n | T(4) ve büyüme | 12; Θ(n log n) |

**Sonucu hesapladıktan sonra çağrı sayısı, derinlik ve maliyeti ayrı kontrol edin.**

# Ödev: rekürans izleyici



İkili arama veya hızlı üs alma için çalışan kod, çağrı izi ve gerekçeli analiz hazırlayın.

| Teslim | İçerik | Kontrol |
|---|---|---|
| rekurans_izleyici.py | Girdi denetimi + algoritma + iz | Normal, sınır ve hata testleri |
| ornek_iz.txt | Girdi / derinlik / dönüş değeri | İlk çağrı ve taban da görünür |
| rapor.md | Taban, bağıntı, açılım, sınır | Çıktı, çağrı ve bellek ayrı |
| Deney karşılaştırması | En az 5 farklı boyut | Ölçülen çağrı ile formül uyumlu |

**Kodlar: kod/cozumlu_ornekler.py ve kod/rekurans_izleme.py • Ayrıntılar: ODEV.md**